Agenda+

Seu dia organizado, serviço por serviço.

Sistema web de agenda de serviços · Spring Boot + JPA + Thymeleaf

Felipe Sperli Neto · **Pedro Henrique Bassi Parma**

Aula 7 — Projeto Agenda de Serviços

CONTEXTO

O problema e o objetivo

O problema

Pequenos negócios de serviços (salões, clínicas, estética) ainda controlam horários em caderno ou planilha. Resultado: horários duplicados, clientes sem contato e nenhum histórico.

O objetivo

Uma aplicação web simples para cadastrar clientes, serviços e profissionais e reservar horários, garantindo as regras de negócio no backend.

- Login com senha criptografada
- Agenda do dia / semana / mês
- CRUD das 6 entidades
- 10 regras de negócio
- Pesquisa com filtros
- Testes automatizados

STACK

Tecnologias utilizadas

Java 17 + Spring Boot 3.5

Aplicação MVC, injeção de dependências e servidor Tomcat embutido.

Spring Data JPA

Repositórios, consultas derivadas e `Specification` para a pesquisa dinâmica.

Banco H2

Banco em arquivo (`./data/agenda`) com console web para mostrar as tabelas.

Thymeleaf + Bootstrap 5

Páginas renderizadas no servidor; Bootstrap via WebJars (funciona offline).

Bean Validation + BCrypt

Validação de campos e senhas gravadas como hash (`spring-security-crypto`).

JUnit 5 + MockMvc

Testes das regras de negócio e das telas, rodando com `mvn test`.

ARQUITETURA

Organização em camadas

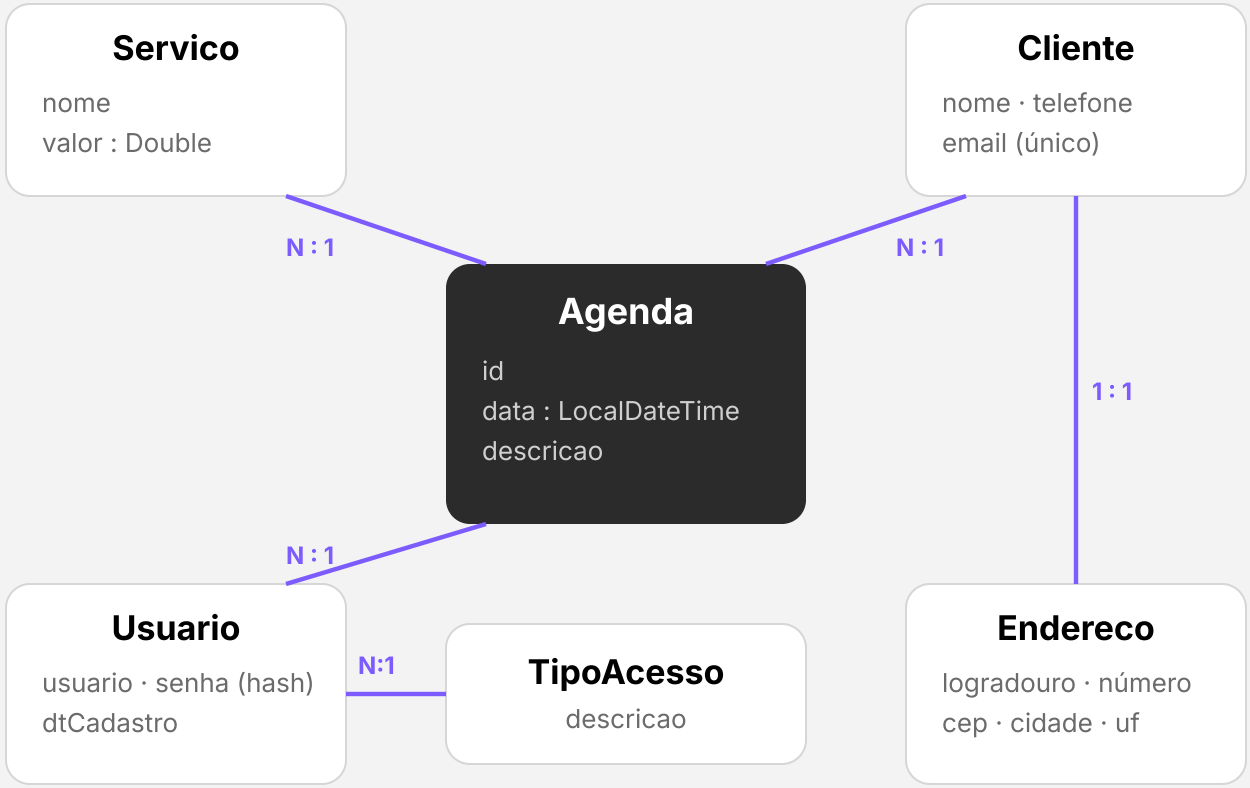


LoginInterceptor protege todas as rotas lança BusinessException → mensagem no campo do formulário

`br.com.agendamais › entity · repository · service · controller · dto · config | resources › templates · static`

MODELO DE DADOS

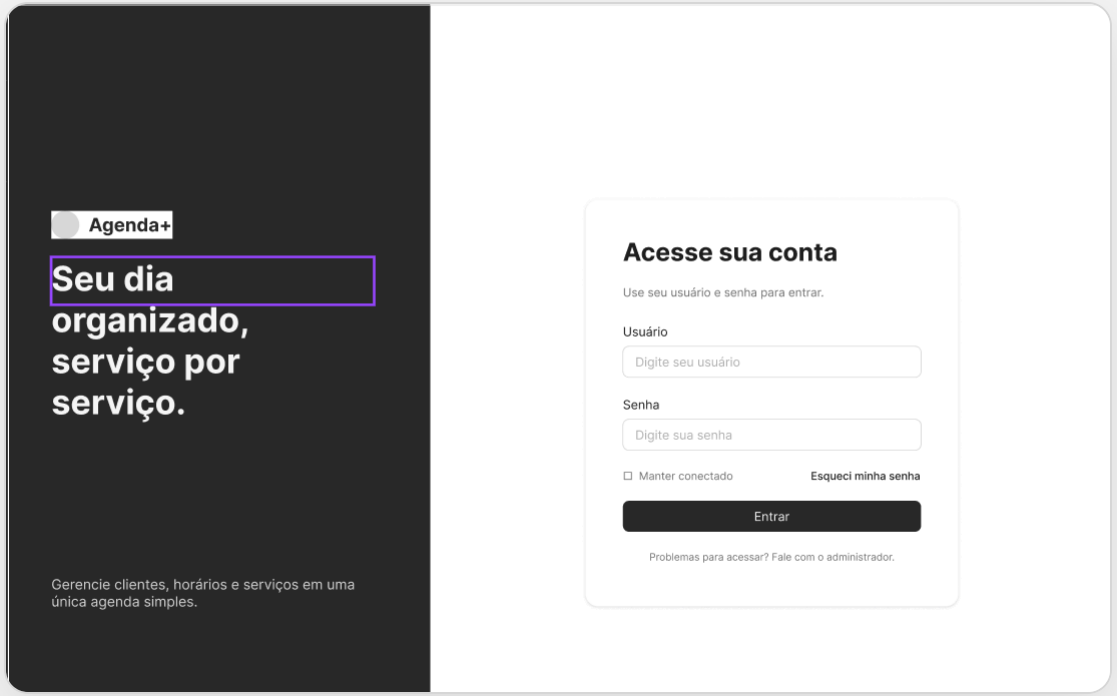
Entidades e relacionamentos (JPA)



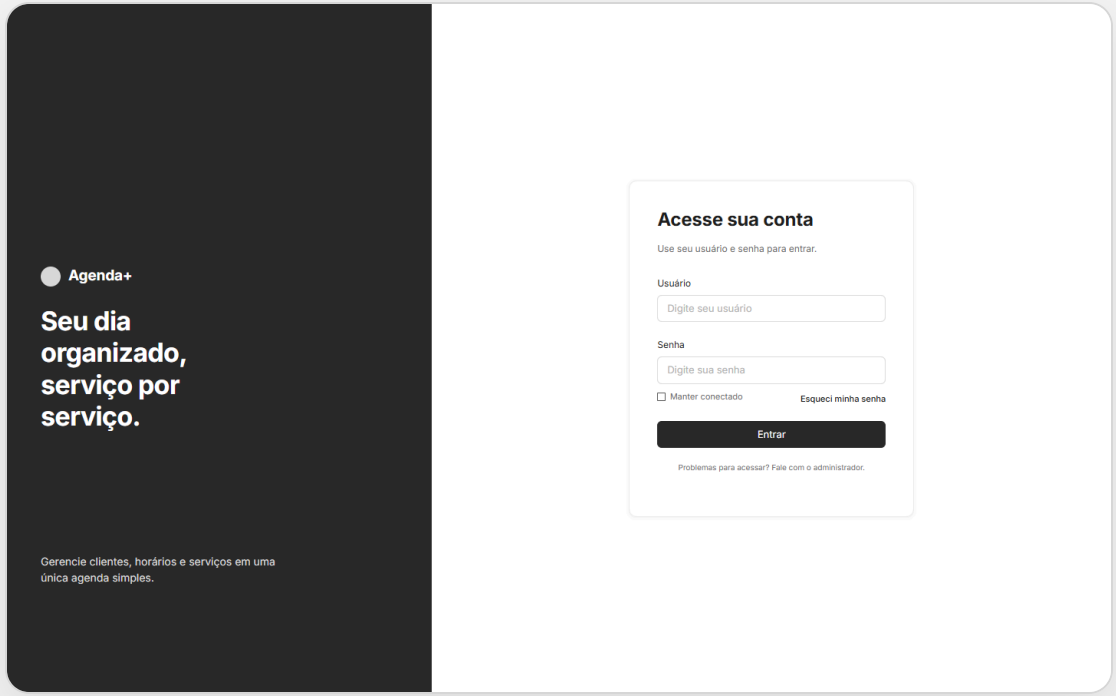
- **Agenda** liga serviço, cliente e profissional (`@ManyToOne` obrigatórios).
- **Cliente** → **Endereco** é `@OneToOne` com cascade e remoção de órfão.
- **Usuario** → **TipoAcesso**: todo usuário tem um perfil (Administrador, Profissional).
- Restrição única `(usuario_id, data)` reforça no banco a regra de conflito de horário.

DO PROTÓTIPO AO SISTEMA

Tela de login: protótipo × implementação



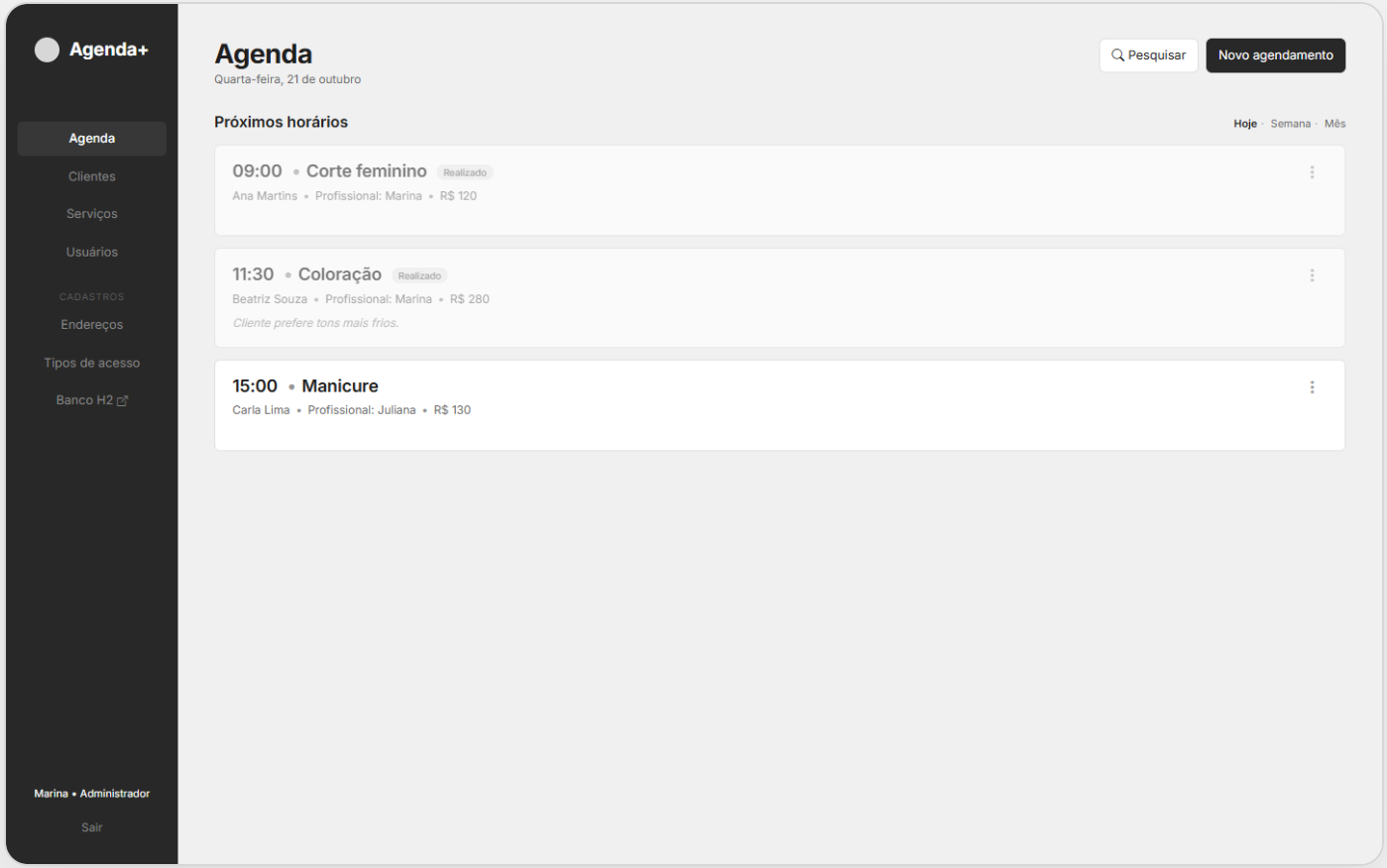
Protótipo das telas



Sistema rodando em `localhost:8080/login`

TELAS

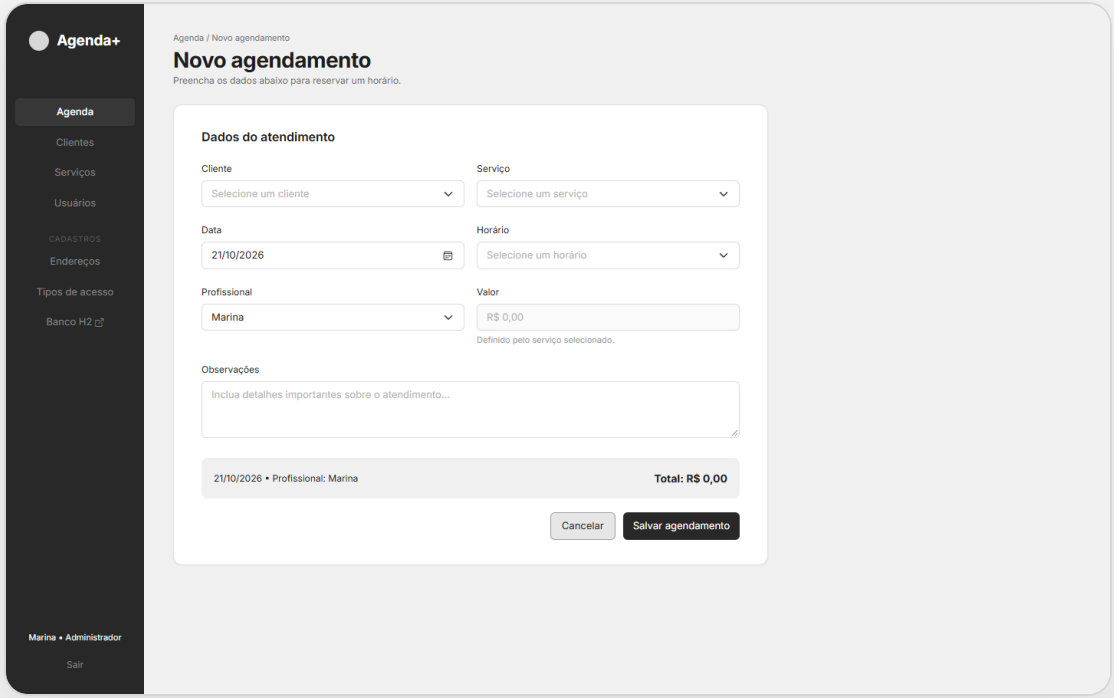
Agenda do dia



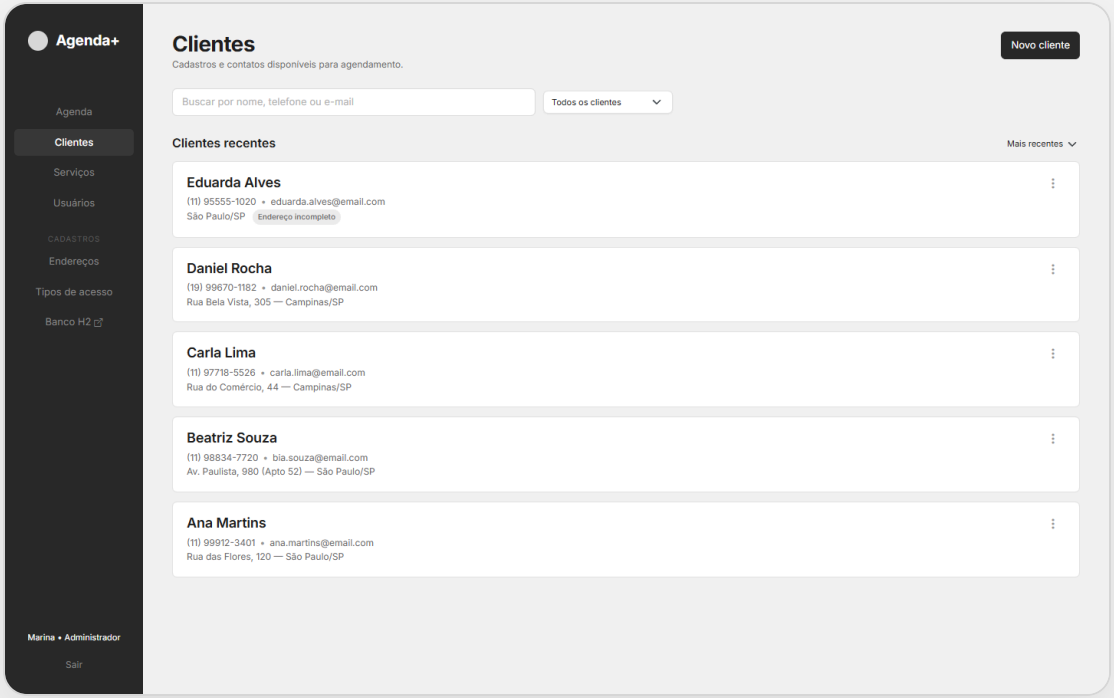
- Visões **Hoje · Semana · Mês**.
- Horários que já passaram aparecem como **Realizado**.
- Botão **Pesquisar** filtra por cliente, serviço e período (RN10).
- Menu ⋮ em cada item: **Editar** e **Excluir** com confirmação.

TELAS

Novo agendamento e clientes



Valor vem do serviço escolhido; horários de 30 em 30 min.



Busca por nome, telefone ou e-mail; filtro de endereço completo/incompleto.

REGRAS DE NEGÓCIO

10 regras, todas na camada Service

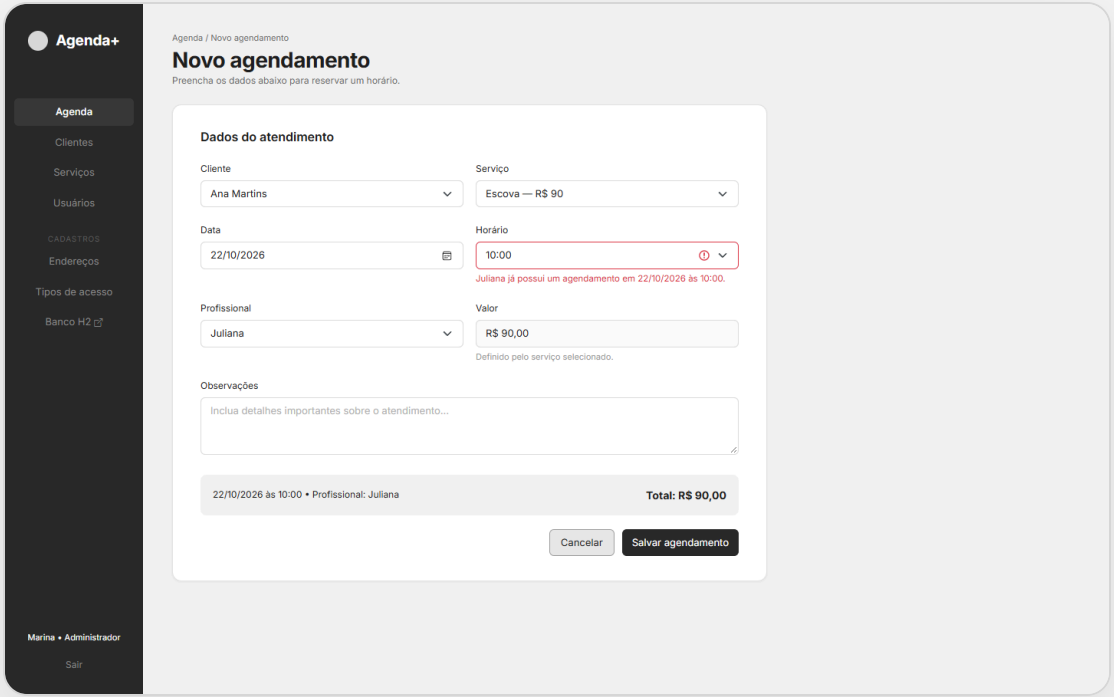
Regra	O que garante	Onde
RN01	Agendamento só para data/horário futuro	`AgendaService.salvar`
RN02	Profissional não pode ter dois atendimentos no mesmo horário	`AgendaService.salvar`
RN03	Valor do serviço deve ser maior que zero	`ServicoService.salvar`
RN04	Agendamento exige cliente cadastrado	`AgendaService.salvar`
RN05	Não exclui serviço vinculado a agendamentos	`ServicoService.excluir`
RN06	Não exclui cliente que possui agendamentos	`ClienteService.excluir`
RN07	E-mail do cliente válido e único	`ClienteService.salvar`
RN08	Usuário precisa de um tipo de acesso válido	`UsuarioService.salvar`
RN09	Senha gravada com hash BCrypt	`UsuarioService.salvar / autenticar`
RN10	Pesquisa por cliente, serviço e período	`AgendaService.pesquisar`

EXEMPLO EM DETALHE

RN02 — conflito de horário

```
boolean conflito = agenda.getId() == null
    ? agendaRepository
        .existsByUsuarioIdAndData(
            usuario.getId(), dataHora)
    : agendaRepository
        .existsByUsuarioIdAndDataAndIdNot(
            usuario.getId(), dataHora,
            agenda.getId());

if (conflito) {
    throw new BusinessException("horario",
        usuario.getNomeExibicao()
        + " já possui um agendamento em "
        + dataHora.format(DATA_HORA) + ".");
}
```



A mensagem vem do **backend**: os formulários usam `novalidate`, então o navegador não bloqueia nada antes.

SEGURANÇA

Login e proteção das páginas

LoginInterceptor

Intercepta todas as rotas. Sem usuário na sessão, redireciona para `/login`.

Senhas com BCrypt

A senha nunca é salva em texto puro. No H2 aparece como `$2a$10$...`; o login usa `passwordEncoder.matches`.

Sessão

“Manter conectado” estende a sessão por 7 dias. Botão **Sair** invalida a sessão.

Perfis de acesso

Usuários de exemplo: **marina** (Administrador) e **juliana** (Profissional). Um usuário não pode excluir a si mesmo, nem quem é responsável por agendamentos.

QUALIDADE

Testes automatizados

16

testes automatizados

0 falhas · 0 erros

Rodando `mvn test` com banco H2 em memória.

RegrasDeNegocioTest — 12 testes

Um ou mais testes por regra, RN01 a RN10. Ex.: `rn02_outroProfissionalPodeUsarOMesmoHorario`, `rn09_senhaGravadaComBCrypt`.

TelasTest — 4 testes (MockMvc)

Login correto, senha errada, redirecionamento sem login e abertura de todas as telas.

DECISÕES DE PROJETO

Ajustes para seguir o diagrama de classes

- “Profissional” no agendamento é o **Usuario** da Agenda.
- “Valor” vem do **Servico** escolhido — a Agenda não tem campo valor.
- “Observações” é o campo `descricao` da Agenda.
- `Agenda.data` é `LocalDateTime`: data **e** horário juntos.
- Horários configuráveis em `application.properties` (08:00–18:00, a cada 30 min).
- Telas extras (Serviços, Usuários, Tipos de acesso, Endereços) completam o CRUD no mesmo estilo.

Como executar

`mvn spring-boot:run`
ou rodar `AgendaServicosApplication` na IDE

Acesso

`localhost:8080`
marina / 123456 (Administrador)

Dados de exemplo

Criados na 1ª execução: 5 clientes, 5 serviços e 6 agendamentos (3 para hoje).

DEMONSTRAÇÃO AO VIVO

Vamos ver o Agenda+ funcionando.

Login → agenda do dia → novo agendamento → regras de negócio → banco H2

Obrigado! · Perguntas?

`github.com/felipeuniara/agenda-servicos`